

AI-Assisted Support Ticket Resolution System

A lightweight requirements + architecture doc. Companion to the client brief; this is the solution side.

1. Purpose & Problem Statement

Support tickets in logistics/supply chain operations are currently triaged manually, even when the answer already exists in prior resolved tickets or documentation. This wastes admin time, produces inconsistent answers, and gives no visibility into recurring root causes.

This system solves: automatic answering of tickets we can answer reliably, human review for everything else, and a knowledge base that grows and improves over time — without ever letting an unreviewed, low-confidence answer reach a customer.

2. Users

User	What they do with the system
End customer	Sends a support ticket, receives a resolution (auto or admin-approved)
Admin / support agent	Reviews low-confidence drafts, edits, approves, sends
Operations manager	Views dashboard — ticket volume, recurring issues, system performance
System owner (agency/dev)	Configures the confidence threshold, monitors system health

3. Data Layer

- **Source data:** existing historical tickets and documentation, provided as JSON/CSV.
- **Ingestion (one-time + ongoing):**
 1. Clean and normalize raw JSON/CSV records (ticket text, resolution text, category, timestamp).
 2. Generate vector embeddings for each record.
 3. Store embeddings + metadata (ticket ID, category, resolution, date) in ChromaDB.

- **Ongoing updates:** every newly resolved ticket (auto-answered or admin-approved) is embedded and added to ChromaDB the same way — this is the "learning loop." The knowledge base is never static.
- **Structured metadata alongside vectors:** category, resolution status, confidence score at time of resolution, whether it was auto-sent or admin-edited — this is what powers the dashboard, not just the vector search itself.

4. Retrieval

- Incoming ticket text is embedded using the same method as the knowledge base.
- ChromaDB is queried for the top-k most similar past tickets/KB entries.
- Retrieved entries are passed as context for answer generation (retrieval-augmented generation pattern) — the system answers using what it actually finds, not from unconstrained memory.

5. Confidence Scoring & Routing

- A confidence score is produced by combining: (a) how close the retrieved matches are to the incoming ticket, and (b) whether the retrieved context is actually sufficient to answer the specific question asked — not just topically similar.
- Threshold is configurable, not hardcoded — the system owner can adjust it based on observed accuracy.
- **Above threshold:** system generates a final answer and sends it directly.
- **Below threshold:** system generates a draft, flags the ticket, notifies the admin (email/Slack), and holds it for review — nothing is sent to the customer until a human approves it.

6. Admin Review Flow

- Admin sees pending tickets in a review queue: original ticket text, retrieved context used, drafted answer, confidence score.
- Admin can edit the draft freely before sending.
- On send, the final (possibly edited) version is what gets emailed to the customer and what gets written back into the knowledge base — not the original unedited draft. This matters: it means the system learns from the *corrected* answer, not its own mistake.

7. Dashboard

- Ticket volume over time
- Auto-answered vs. admin-reviewed split
- Confidence score distribution
- Admin edit rate (how often drafts are changed before sending — the clearest signal of real system accuracy)
- Recurring categories/topics — the pattern-surfacing layer that's the actual differentiator for the client, not just the automation itself
- Pending queue view

8. Tech Stack

Layer	Choice
Backend API	Python (FastAPI)
Vector database	ChromaDB
Relational store	Postgres (tickets, metadata, audit trail)
Embedding + generation	LLM API (provider TBD per client constraints/budget)
Async processing	Task queue (e.g. Celery/RQ) — handles ticket spikes without blocking
Frontend/dashboard	React
Email integration	SMTP or helpdesk provider API (Gmail API / Zendesk / etc., depending on client's existing tools)
Notifications	Email or Slack webhook to admin

9. Non-Functional Requirements

- Low-confidence tickets must never bypass admin review — this is a hard rule, not a tunable one.
- Every send (auto or admin-approved) is logged with the confidence score and, for admin-reviewed ones, the diff between draft and sent version — this is the audit trail.
- Threshold changes should not require a redeploy — expose it as a config value, not a hardcoded constant.

10. Future Work

- Multi-channel intake (WhatsApp, chat widget) beyond email
- Multi-language support
- Feedback signal from customers (thumbs up/down on auto-answers) to refine confidence scoring over time, not just admin edit rate
- Auto-suggested threshold tuning based on observed admin edit rates
- Priority/severity detection on top of category classification (route urgent exceptions faster)
- Root-cause analytics — not just "what category" but "which carrier/route/vendor is driving this category," closing the loop back to the client's actual operational decisions

11. Scaling Considerations

- **Vector DB:** ChromaDB is fine at pilot/single-client scale; if ticket/KB volume grows large or this becomes multi-tenant (multiple clients on one system), plan a migration path to a managed vector DB (e.g. Pinecone/Weaviate/pgvector) rather than assuming Chroma indefinitely.
- **Processing:** async task queue prevents ticket spikes from blocking intake; can scale worker count horizontally as volume grows.
- **Multi-tenancy:** if this becomes a reusable product across multiple clients (not just one logistics company), the data layer needs tenant isolation from day one — mixing knowledge bases across clients would be a serious mistake, not just a scaling detail.
- **Cost control:** LLM calls for generation should be monitored — high ticket volume means embedding + generation costs scale linearly; batch embedding for the initial KB load reduces one-time cost, and caching repeated/near-identical queries reduces ongoing cost.